

Mobile-ShoppingAgent 系统架构

版本: v4.1 (AMSG Runtime)

更新: 2026-05-28

1. 要解决什么问题

当前 Mobile GUI Agent 的核心瓶颈:

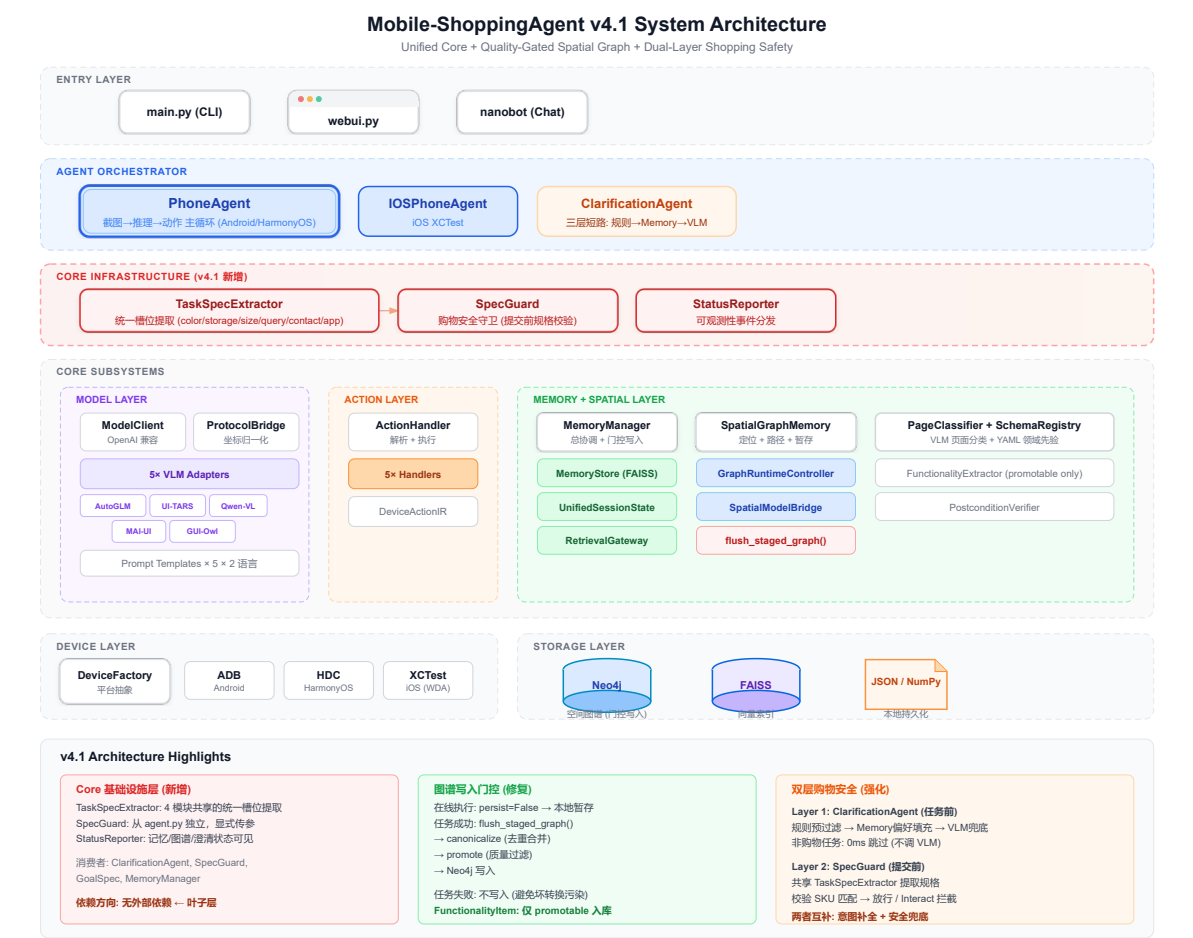
瓶颈	我们的方案
每步都依赖 VLM, 延迟高、成本大	空间图谱导航: 已知路径跳过 VLM, 毫秒级执行
跨会话记忆缺失, 每次从零开始	三层记忆架构: 向量语义 + 图结构 + 会话状态
购物操作不可逆, 误下单/选错规格	ClarificationAgent (任务前补全) + SpecGuard (提交前校验)

核心理念——质量可控的自进化闭环:

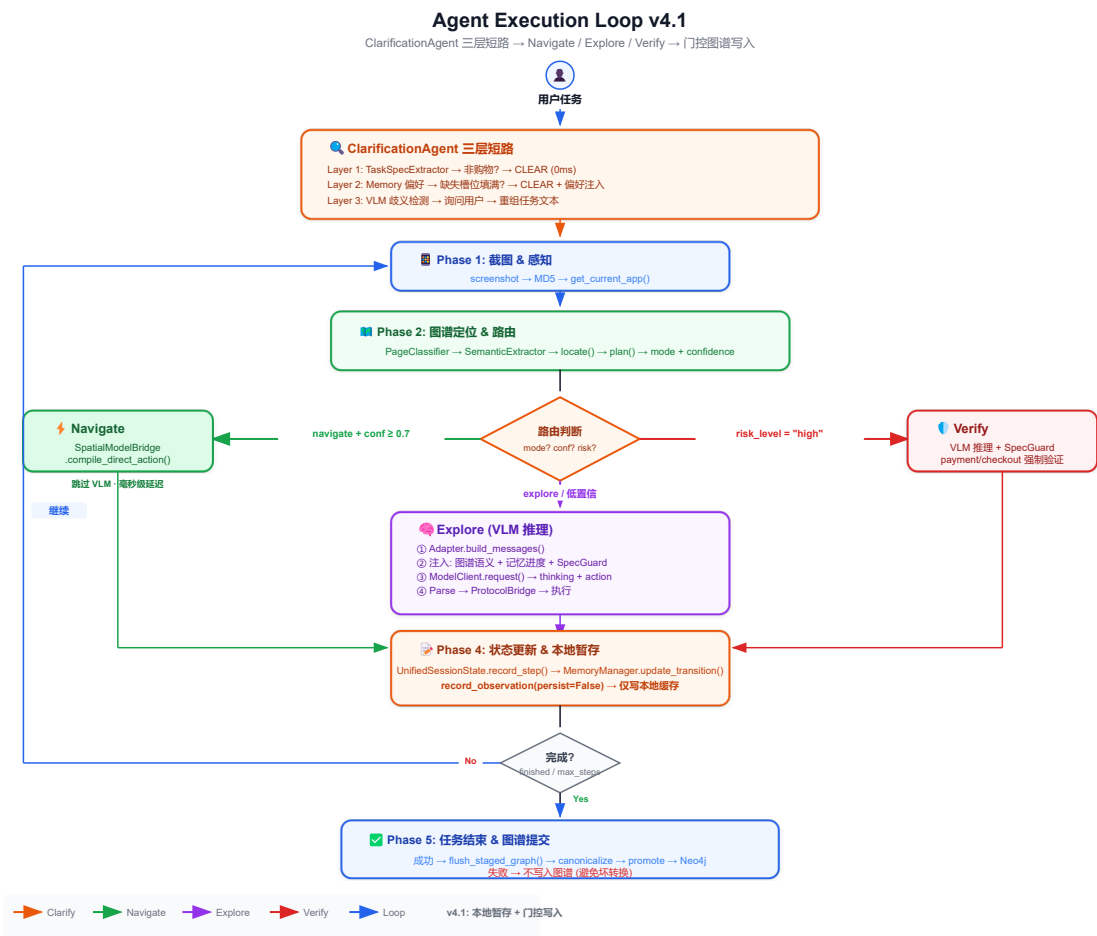
传统: 截图 → VLM → 动作 → 截图 → VLM → 动作 → ... (每步依赖 VLM)

我们: 截图 → 图谱定位 → 已知路径 → 图谱导航 → 直接执行 (跳过 VLM)
↳ 未知场景 → VLM → 执行 → 本地暂存 (成功后审核入库)

2. 系统架构



3. Agent 主流程



Agent 每步执行遵循 **感知** → **定位** → **路由** → **执行** → **记录** 的循环：

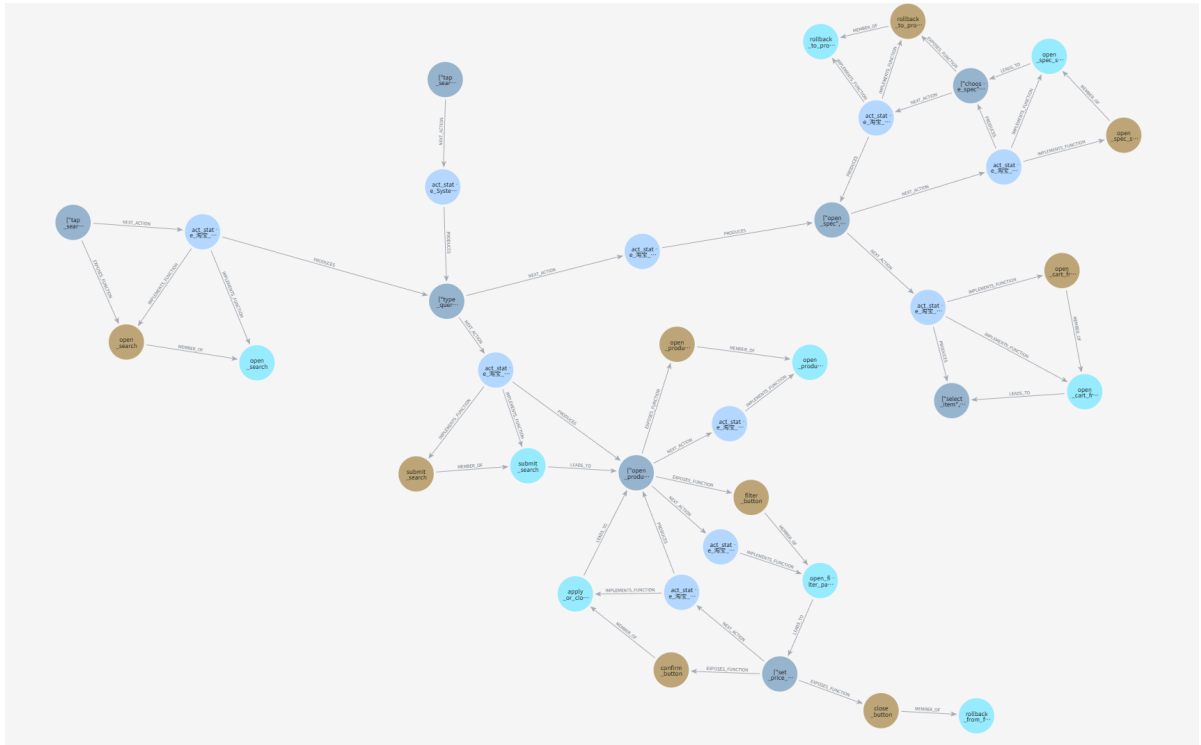
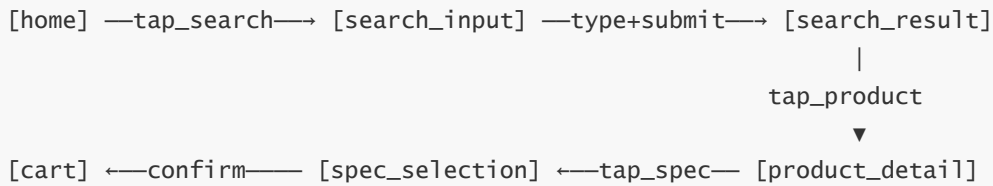
1. **ClarificationAgent**（首步）：三层短路检测任务歧义——规则提取 → Memory 偏好填充 → VLM 兜底。非购物任务 0ms 跳过。
2. **感知**：截图 → PageClassifier 分类页面类型
3. **定位**：SpatialGraphMemory.locate() 在图谱中定位当前页面
4. **路由**：GraphRuntimeController 根据置信度和风险分级决定三种模式
 - **Navigate**：图谱有高置信度路径 → 跳过 VLM，直接执行
 - **Explore**：未知场景 → VLM 推理，同时本地暂存观测
 - **Verify (verify_with_vlm)**：高风险页面（支付/结算）→ 即使有路径也强制 VLM + SpecGuard 验证
5. **记录**：record_observation(persist=False) 仅写本地缓存；任务成功后 flush_staged_graph() 经去重和过滤才入库

4. 空间图谱 (AMSG)

4.1 图谱是什么

AMSG (Active Mobile Spatial Graph) 是一个**语义级的应用导航图谱**。与 UI 元素树或像素特征不同，它工作在功能语义层——关注的是“这个页面能做什么、怎么到下一个页面”。

AMSG 图谱示例（淘宝购物流）



4.2 节点设计：PageNode

每个节点代表一个**语义页面状态**。核心标识是 `page_type`（如 home、search_result、product_detail），描述“这是什么页面”；辅以 landmarks 和 affordances 描述“页面上有什么、能做什么”。

节点关键属性：

属性	设计意义
page_type	节点的主标识。18 种语义类型（home, search_input, search_result, product_detail, spec_selection, cart, checkout, payment, address, filter_panel, category, my_account, settings, store, login, dialog, permission, unknown）
landmarks	看到什么 ——页面上可观测的关键 UI 要素（搜索框、商品卡片、价格标签、筛选栏）。来自两个来源的合并：Schema YAML 先验知识 + 运行时 PageClassifier 从截图提取的 elements
affordances	能做什么 ——该页面提供的操作能力（open_search、open_product、add_to_cart、filter）。同样由 Schema 先验 + 运行时推断合并而来
slots	当前观测到的动态值（query="iPhone 17"、price="¥8999"）。注意：这是 观测值 ，不是模板参数

属性	设计意义
risk_level	安全分级，由 Schema 中的 page_type 映射决定：normal → 可直接导航，medium → 降低置信度阈值，high (checkout/payment/login) → 强制 VLM 验证
semantic_signature	去重键，由 app domain page_type landmarks affordances slots 拼接后哈希。同一语义页面在不同截图间产生相同签名 → 合并为一个节点，避免变体膨胀

landmarks vs affordances 的区别：landmarks 回答"页面长什么样"（视觉锚点），affordances 回答"用户下一步能做什么"（动作空间）。图谱路由时，affordances 决定从当前页面可以走哪些边；landmarks 用于定位匹配（当前截图是否对应这个节点）。

对 UI 改版鲁棒的原因：节点的身份不是像素或坐标，而是 page_type + 语义签名。淘宝改了首页布局，只要"搜索框 + 商品推荐"的语义结构不变，节点仍然匹配。

4.3 边设计：TransitionEdge

每条边代表一次**页面间的语义转换**，携带：

属性	作用
action_type	语义意图 (click, type_text, scroll, go_back)
action_target	操作对象 ("搜索框", "加入购物车按钮")
postcondition	执行后预期到达的页面类型
confidence	可靠性，基于历史成功/失败统计自动更新
cost	路径规划权重：1.0 + fail_rate×3 + risk_penalty - confidence×0.3

此外还携带 risk（安全分级）、success_count / fail_count（执行统计）、rollback_action（回退动作）、evidence（来源证据）等运行时元数据。

边的 semantic_edge_key（由 source.page_type|intent|semantic_target|region|target.page_type 组成）用于去重——同一语义转换不会因为坐标微小偏移产生重复边。

特殊的**复合边 (Compound)**：将"输入关键词 → 提交搜索"合成为一条带 <query> 槽位的模板边，运行时填入实际搜索词即可复用。

4.4 Agent 如何检索图谱获得路线

当 Agent 收到一个任务（如"去淘宝买 iPhone 17 Pro Max 银色 512G，加入购物车"），图谱检索分三步：

Step 1: 目标推断 (GoalSpec)

从任务文本提取目标页面类型和槽位：

- "加入购物车" → 目标页面 = spec_selection（加购按钮在规格选择弹窗上）
- "iPhone 17 Pro Max 银色 512G" → slots = {query, color, storage}

Step 2: 当前定位 (locate)

将当前截图的 PageClassifier 分类结果与图谱中的节点匹配：

- 输入：当前 app=淘宝, page_type=home, semantic_signature
- 输出：PageBelief (最佳匹配节点 + 置信度 0.92)

Step 3: 路径规划 (plan)

在图谱上执行 Dijkstra 最短路径搜索：

- 起点：当前节点 (home)
- 终点：目标页面类型 (spec_selection)
- 权重：edge.weighted_cost (综合成功率、风险、置信度)
- 输出：RoutePlan (mode=navigate, 路径=[home → search_input → search_result → product_detail → spec_selection], 下一步动作)

如果置信度 ≥ 0.7 且 risk ≠ high, Agent 直接执行图谱给出的动作 (Navigate 模式), 跳过 VLM 推理。否则回退到 VLM (Explore 模式), 同时将新观测暂存到本地缓存供未来使用。

4.5 功能层：FunctionalityItem 与 FunctionalityCluster

图谱的节点和边构成了导航层 ("怎么从 A 到 B")。在此之上，还有一个功能层，回答"当前页面上有哪些已验证的操作能力"。

4.5.1 FunctionalityItem (功能节点)

每个 FunctionalityItem 是从页面观测中提取的一个语义操作原语，不是图谱节点，而是附着在 PageState 上的标注：

属性	含义
functionality_id	稳定标识, 由 hash(app, page, label, description) 生成
type	"functionality" (可操作: 加购、搜索) 或 "data" (观测值: 价格、标题)
canonical_role	语义角色 (open_search、add_to_cart、submit_search)
is_promotable	入库门控标志。仅 type=functionality 且有 canonical_role 的项为 True
observed_postcondition	执行后实际到达的页面类型 (来自历史验证)
confidence	置信度 (0.5-0.9)

关键区分：type=data 的项 (具体价格、商品标题) 标记 is_promotable=False, 永远不会入库——避免瞬态数据污染图谱。

4.5.2 FunctionalityCluster (功能簇)

将语义相似的 FunctionalityItem 聚合为簇，代表一类可复用的操作能力：

属性	含义
cluster_id	由 canonical_role + postcondition 派生

属性	含义
canonical_name	簇的语义名称（继承自代表性成员的 role）
member_functionality_ids	所有成员 FunctionalityItem 的 ID 集合
page_types	该簇出现在哪些页面类型上
verified_edges	已验证成功的转换签名
success_count / fail_count	聚合的执行结果统计

聚类算法：基于 `functionality_similarity()` 函数（阈值 0.58），优先匹配 canonical_role（0.95 权重），其次匹配 page_type + postcondition（0.9），最后基于 token Jaccard 重叠度。

4.5.3 功能层在运行时的作用

功能层**不参与** 4.4 节的导航路由（locate → plan → route），而是作为 **Explore 模式下 VLM 推理的语义增强层**：

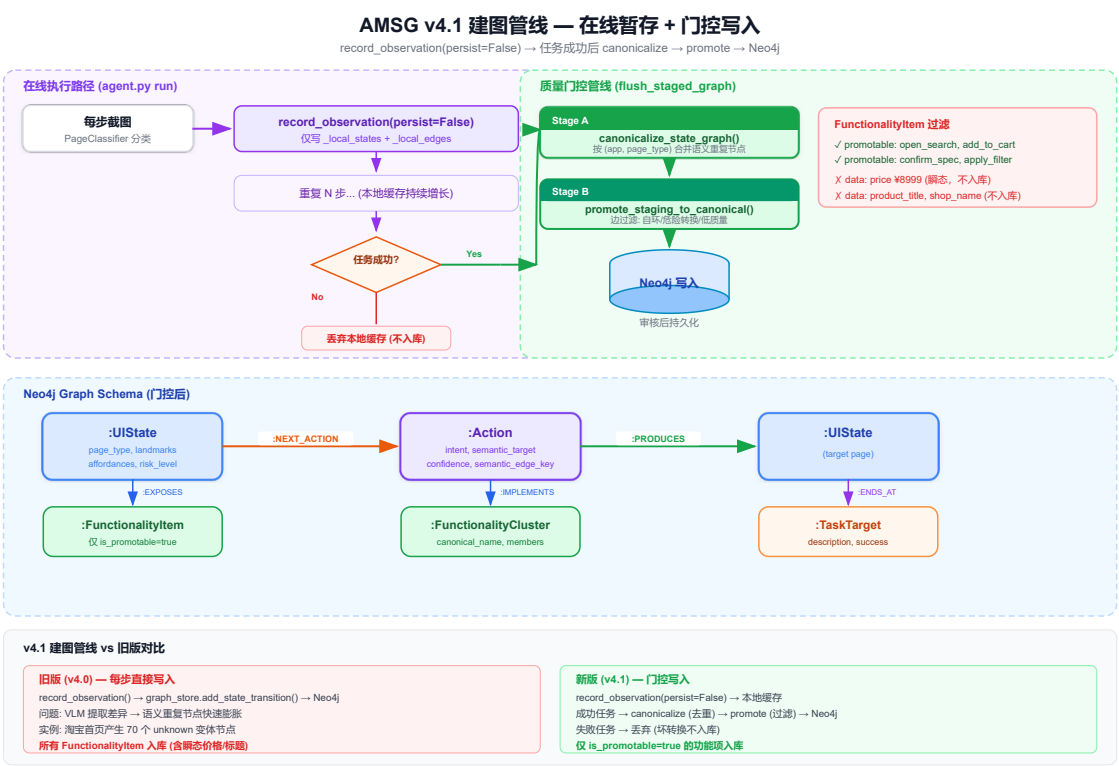
```
GraphRuntimeController.decide_next()
├─ 导航层: locate → plan → route (决定 navigate/explore/verify_with_vlm)
└─ 功能层: _load_functionality_context()
    → graph_store.get_v4_functionality_context()
        → 查询当前页面的 FunctionalityItem (is_promotable=true)
        → 查询匹配的 FunctionalityCluster (success_count > 0)
    → _inject_v4_hint() → 作为 [v4 knowledge] 注入 VLM prompt
    → _enrich_next_action_with_functionality() → 补充动作语义信息
```

为什么不直接参与路由：导航层已经通过 TransitionEdge 的 cost 权重覆盖了"怎么走"的问题。功能层解决的是另一个问题——当 Agent 进入未知页面（Explore 模式）需要 VLM 推理时，告诉 VLM"这个页面上历史验证过哪些操作是有效的"，减少 VLM 的试错空间。

4.5.4 入库管线与质量门控

```
页面 + 转换 → FunctionalityExtractor.from_page() → FunctionalityItem[]
→ FunctionalityClusterer.cluster() (过滤 is_promotable=False)
→ FunctionalityQualityGate.evaluate() (验证饱和度、新颖性、风险平衡)
→ graph_store.upsert_functionality_graph() → Neo4j
    └─ (UIState)-[:EXPOSES_FUNCTION]->(FunctionalityItem)
        └─ (FunctionalityCluster)-[:CONTAINS_FUNCTION]->(FunctionalityItem)
```

4.6 建图管线



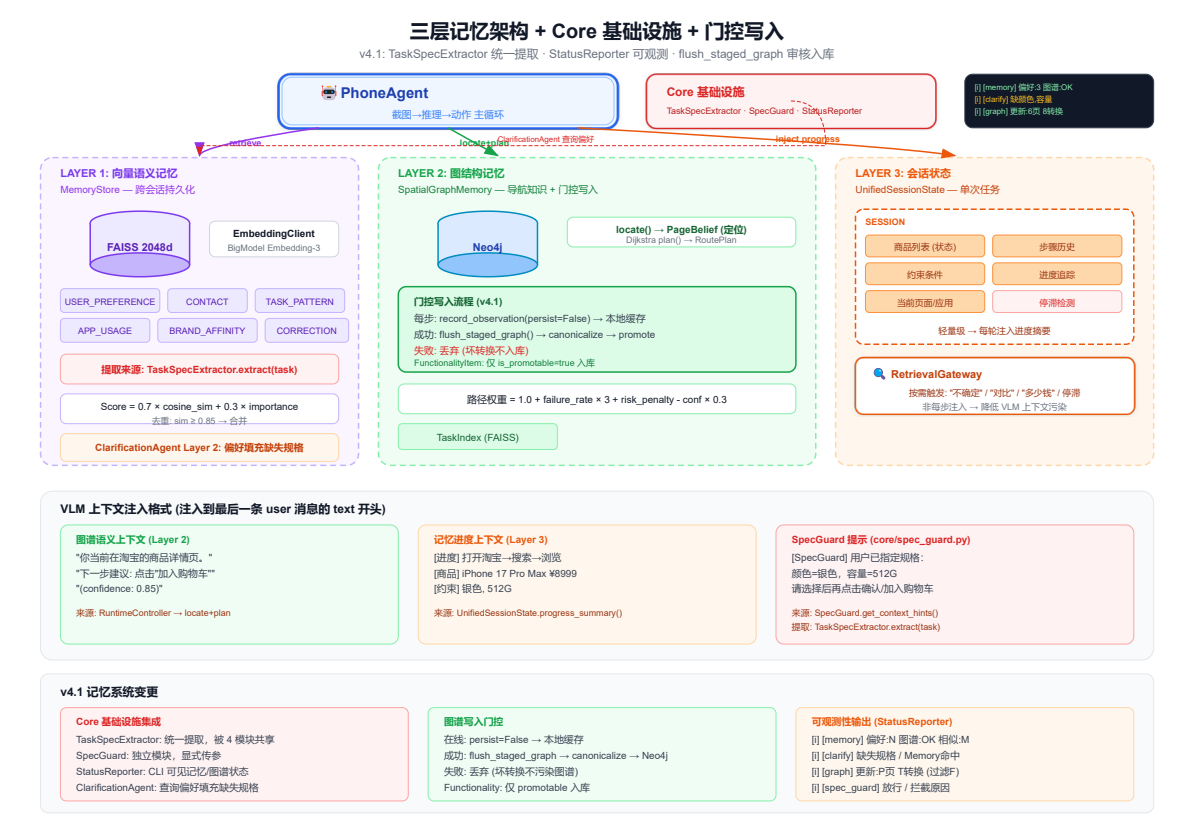
5 阶段管线: 截图 → PageClassifier 分类 → 语义提取 → 节点去重 → 边构建 → 功能发现

4.7 图谱质量门控 (v4.1)

写入路径	策略	门控
在线执行	每步 <code>persist=False</code> → 仅本地缓存	不直接写 Neo4j
任务成功	<code>flush_staged_graph()</code> → <code>canonicalize</code> → <code>promote</code>	去重 + 质量过滤
任务失败	丢弃	坏转换不入库

功能发现过滤: 只有可操作功能 (`open_search` , `add_to_cart`) 入库, 瞬态数据 (具体价格、商品标题) 不入库。

5. 全局记忆系统



三层架构, 各解决不同问题:

层	存储	内容	生命周期
向量语义	FAISS 2048d	用户偏好、联系人、任务模式	跨会话持久化
图结构	Neo4j (AMSG)	页面状态、转换路径、功能簇	持久化 (门控写入)
会话状态	内存	商品列表、步骤历史、约束	单次任务

关键设计决策:

- 按需检索** (RetrievalGateway): 不是每步都注入记忆, 只在 VLM 表现出不确定 (“忘记了”、“对比”) 或停滞时触发
- ClarificationAgent 接入 Memory**: 规格缺失时先查偏好 (“用户上次买 512G”), 有命中就不打断用户
- 可观测性** (StatusReporter): CLI 可见记忆系统状态——偏好数量、Neo4j 连接、相似任务命中

6. ClarificationAgent + SpecGuard 双层购物安全

两者共享 TaskSpecExtractor 提取基础设施, 但职责互补:

	ClarificationAgent	SpecGuard
时机	任务开始前	购买提交前

	ClarificationAgent	SpecGuard
目的	补全缺失信息 → 提高规划精度	校验 SKU 匹配 → 防止误操作
手段	规则 → Memory → VLM (三层短路)	从 task + VLM plan 提取规格 → 与页面选择比对
结果	重组任务文本	放行 / Interact 拦截

TaskSpecExtractor 统一了此前分散在 4 处的规格提取逻辑 (color/storage/size/query/contact/app) , 被 ClarificationAgent、SpecGuard、GoalSpec、MemoryManager 共享。

7. 解决了 Mobile Agent 的什么问题

当前 Mobile GUI Agent 领域存在四个结构性瓶颈, 每个都有大量工作在尝试解决, 但尚无系统性方案。我们的设计针对这四个瓶颈提出了对应的架构回应:

7.1 推理效率瓶颈: 每步必须调用 VLM

现状: 从 DigiRL (NeurIPS 2024) 到 UI-TARS (2025) 再到 Mobile-Agent-v3 (2025), 所有现有 Agent 的每一步都必须调用 VLM 推理。即使是完全相同的操作序列 (如"打开淘宝 → 搜索 → 选商品"), 每次执行都从零推理。

问题本质: VLM 推理是 Mobile Agent 最大的延迟和成本来源 (单步 1-5s) , 但大量操作是重复的、可预测的。

我们的方案——VLM + 图谱混合导航:

- AMSG 空间图谱将已验证的页面转换路径结构化存储
- GraphRuntimeController 三模式路由: 已知路径走 Navigate (跳过 VLM, 毫秒级)、未知场景走 Explore (VLM 推理)、高风险走 Verify (强制 VLM + SpecGuard)
- 功能层作为 Explore 模式的语义增强, 将已验证的操作能力注入 VLM prompt, 减少试错
- 结果: 已知路径延迟从秒级降至毫秒级, 同时保留未知场景的 VLM 兜底能力

7.2 经验沉淀瓶颈: 跨会话记忆缺失

现状:

- 零记忆派 (DigiRL、CogAgent、OS-Atlas、GUI-Odyssey) : 每次任务从零开始
- 浅层记忆派 (AppAgent v2 的 RAG 文档库、Mobile-Agent-v3 的 Notetaker + RAG) : 有持久化但结构扁平, 无法表达页面间的拓扑关系
- 会话内记忆派 (UI-TARS 的压缩 episode summary) : 不跨会话持久化

问题本质: Agent 的经验无法结构化积累。用户的个人偏好、App 的导航拓扑、单次任务的动态状态, 三者需要不同的存储和检索策略。

我们的方案——三层记忆 + 按需检索:

- 向量语义层 (FAISS 2048d) : 用户偏好、联系人、任务模式 → 跨会话持久化

- 图结构层 (Neo4j AMSG)：页面状态、转换路径、功能簇 → 质量门控持久化
- 会话状态层 (UnifiedSessionState)：商品列表、步骤历史、约束 → 单次任务
- RetrievalGateway 按需检索：不是每步注入全量记忆，只在 VLM 表现出不确定或停滞时触发，避免上下文污染

7.3 安全性瓶颈：高风险操作无防护

现状：在所有主流 Mobile Agent 中，几乎没有系统内建购物安全机制。VeriSafe Agent (MobiCom 2025) 首次提出了基于形式逻辑的预动作验证，但它是一个独立的验证层，不解决意图补全问题。其余系统 (DigiRL、UI-TARS、AutoGLM、AppAgent v2、Mobile-Agent-v3) 均无安全防护。

问题本质：购物场景下"选错规格 → 误下单"是不可逆的。Agent 需要在两个时间点进行拦截——任务开始前（意图是否完整？）和提交操作前（选择是否匹配？）。

我们的方案——ClarificationAgent + SpecGuard 双层防护：

- ClarificationAgent（任务前）：三层短路检测意图歧义（规则提取 → Memory 偏好填充 → VLM 兜底），非购物任务 0ms 跳过
- SpecGuard（提交前）：从 task + VLM plan 提取目标规格，与页面实际选择比对，不匹配则拦截
- TaskSpecExtractor 统一提取基础设施，被两者 + GoalSpec + MemoryManager 共享，避免重复解析

7.4 图谱质量瓶颈：在线学习的噪声控制

现状：AutoGLM (2024) 和 DigiRL (2024) 都采用在线学习，但它们的学习信号是模型权重级的（RL reward → 参数更新），不涉及结构化知识的质量控制。AppAgent v2 的 RAG 文档库无去重机制。没有现有工作解决"在线图谱写入如何避免噪声节点污染"的问题。

问题本质：无门控的在线写入会导致图谱膨胀（实测：70 个 unknown 节点 → 去重后 7 个）。瞬态数据（具体价格、商品标题）如果入库，会产生大量无法复用的节点。

我们的方案——质量门控 + 功能发现过滤：

- 在线执行时 `persist=False`，仅写本地缓存
- 任务成功后 `flush_staged_graph()` 经 `canonicalize` → `promote` 管线入库，任务失败则丢弃
- FunctionalityExtractor 区分 `functionality`（可操作）和 `data`（观测值），`is_promotable` 标志控制只有有 `canonical_role` 的功能项入库
- FunctionalityQualityGate 评估验证饱和度、新颖性、风险平衡

与前沿工作对比

选取 2024-2025 年 6 个代表性系统，覆盖 RL 路线 (DigiRL、AutoGLM)、端到端路线 (UI-TARS)、多智能体路线 (Mobile-Agent-v3)、知识增强路线 (AppAgent v2) 和安全路线 (VeriSafe)：

维度	DigiRL	AppAgent v2	UI-TARS	AutoGLM	Mobile-Agent-v3	VeriSafe	Ours
	NeurIPS'24	2024.08	2025.01	2024.10	2025.08	MobiCom'25	
每步 VLM	必须	必须	必须	必须	必须	N/A (验证层)	可选 (Navigate 跳过)
跨会话记忆	无	RAG 文档库 (扁平)	无 (会话内 summary)	无	Notetaker + RAG	无	三层结构化 (向量+图谱+会话)

维度	DigiRL	AppAgent v2	UI-TARS	AutoGLM	Mobile-Agent-v3	VeriSafe	Ours
知识表示	无	扁平文档	无	无 (RL 权重)	RAG 外部知识	形式逻辑规约	语义图谱 (PageState + TransitionEdge + FunctionalityCluster)
图谱质量控制	N/A	无去重	N/A	N/A	N/A	N/A	门控写入 + promotability 过滤
安全机制	无	无	无	无	无	预动作逻辑验证	双层 (意图补全 + SKU 校验)
多模型	单模型 (1.3B)	GPT-4V	单端到端模型	GLM 系列	商用 MLLM	可插拔	5 种 VLM + 坐标归一化
多平台	Android	Android	桌面+移动+Web	Web+Android	Android+桌面	Android	Android + HarmonyOS + iOS
学习方式	离线→在线 RL	探索→部署 (两阶段)	端到端预训练	在线课程 RL	多智能体协作	规约自动化	在线图谱进化 (任务成功→结构化入库)

核心差异化：

- 1. **推理可选性**：所有现有 Agent 每步必须 VLM，我们是唯一支持已知路径零 VLM 推理的系统
- 2. **结构化经验**：现有记忆方案要么没有（DigiRL、UI-TARS），要么是扁平 RAG（AppAgent v2、Mobile-Agent-v3）。我们的 AMSG 是目前唯一将 App 导航经验建模为带质量门控的语义图谱的方案
- 3. **安全覆盖度**：VeriSafe 只做预动作验证 ("动作是否合法")，不做意图补全 ("用户是否说清楚了")。我们的双层设计覆盖了购物安全的完整链路
- 4. **功能层语义增强**：FunctionalityCluster 为 VLM 提供"这个页面上历史验证过哪些操作"的先验知识，这在所有现有工作中没有对应设计

值得深挖的方向

- 1. **图谱预训练**：大规模 App 截图预训练通用 AMSG，解决新 App 冷启动问题
- 2. **自适应置信度**：根据用户风险容忍度和任务类型动态调节 Navigate/Explore 切换阈值
- 3. **图谱压缩与蒸馏**：保持导航质量前提下的图谱蒸馏，支撑端侧部署
- 4. **VLM 微调反馈环**：图谱成功/失败统计作为 RLHF 信号，形成 VLM ↔ 图谱协同进化
- 5. **离线-在线融合**：OfflineExplorer 预探索与在线增量更新的优雅融合策略